

Guía de trabajo del equipo — Scaps

Cómo trabajamos y qué hay para hacer en el Sprint 0

Scaps es nuestro e-commerce de gorras con visor de productos en 3D. Esta guía explica, en pocos minutos, cómo nos organizamos día a día y cuáles son las tareas del primer sprint. Léela antes de tomar tu primera tarea.

Cómo nos organizamos

Trabajamos con un tablero de tareas en **GitHub Projects**, no con roles fijos. No hay "dueños" de áreas: el trabajo está en un backlog priorizado y **cada uno toma la tarea que sigue**. Una tarea avanza por estas cinco columnas, de izquierda a derecha:



- **Backlog** — todo lo que hay que hacer pero todavía no entró al sprint.
- **To Do** — listo para tomar. Tareas priorizadas que ya se pueden agarrar.
- **In Progress** — lo que alguien está haciendo ahora. Una sola por persona.
- **In Review** — terminado, esperando que otro lo revise (Pull Request).
- **Done** — hecho, revisado y mergeado.

El flujo, paso a paso

Cuando te vas a poner a trabajar:

1. **Entrás al tablero** (GitHub Projects) y mirás la columna **To Do**.
2. **Tomás la tarea de más arriba**. Están ordenadas por prioridad: no elijas la más fácil del medio.
3. **La movés a In Progress** y la **convertís en issue** (clic en la tarjeta → *Convert to issue*). Le ponés los **labels** que figuran en el backlog (área + prioridad) y **te la asignás**, para que el resto vea que la estás haciendo.
4. **Creás una rama** desde `main` para esa tarea (por ejemplo `feat/scaffold-frontend`).
5. **Programás** y vas guardando tus commits en esa rama.
6. Cuando terminás —y cumple el "*hecho cuando*"— **abrís un Pull Request** y en la descripción escribís `Closes #N` (el número del issue).
7. **Otro del equipo lo revisa** y aprueba. Si pide cambios, los hacés y volvés a pedir revisión.
8. **Se mergea el PR**. El issue se cierra y la tarjeta **pasa sola a Done**: no tenés que mover nada a mano.

Las reglas (pocas y simples)

- **Una sola tarea en curso por persona**. Terminá lo que empezaste antes de tomar otra.
- **Tomá de arriba hacia abajo** en To Do. Lo prioritario primero.
- **No se pushea directo a `main`**. Todo entra por Pull Request con al menos una aprobación.

- **Si te trabás más de un día**, marcá la tarjeta como bloqueada y pedí ayuda. Una tarea crítica frenada en silencio es lo peor que puede pasar.
- **El que revisa** controla que la tarea cumpla su *"hecho cuando"* y que no rompa nada.

Backlog del Sprint 0

El Sprint 0 es preparación: dejar el proyecto listo para empezar a construir. Cada tarea indica su área entre corchetes, qué hacer concretamente y, cuando aplica, el criterio para darla por terminada (*hecho cuando*).

To Do — se pueden tomar ya:

- **[visor-3d] Conseguir modelos .glb de un banco gratuito** — bajar 3 o 4 modelos 3D en formato .glb de un banco gratuito (Sketchfab, Poly Pizza) para usarlos como productos de demo. Licencia **CC0** (uso libre, sin atribución); idealmente que parezcan gorras. *Hecho cuando*: los .glb están en el repo o accesibles, listos para el catálogo y el POC.
- **[setup] Inicializar el monorepo con workspaces** — crear `apps/web` (frontend) y `apps/api` (backend) y un `package.json` raíz con la clave `"workspaces": ["apps/*"]` (le avisa a npm que las dos apps van juntas). Sin `packages/shared`, sin Turborepo ni Nx. *Hecho cuando*: `npm install` desde la raíz instala los dos workspaces y está en `main`.
- **[infra] Proteger la rama main** — en GitHub, **Settings** → **Branches** → **Add rule**: activar "Require a pull request before merging" con 1 aprobación, para que nadie pushee directo a `main`. (*Configuración en GitHub.*)
- **[setup] Diseñar el modelo de datos (ERD)** — definir las tablas/entidades del MVP (usuarios, roles, productos con "destacado", imágenes y modelo .glb, stock, carrito, órdenes), sus campos y relaciones. Se puede usar dbdiagram.io o draw.io. *Hecho cuando*: hay un ERD acordado y guardado en `docs/`.
- **[setup] Definir el contrato de la API** — listar los **endpoints** (direcciones de la API) con método, ruta, request y response. Ejemplo: `GET /products` → lista de productos. Es lo que permite a front y back ir en paralelo. *Hecho cuando*: documento acordado en `docs/`. (*Depende del ERD.*)
- **[setup] Diseñar los wireframes** — bocetos simples (solo dónde va cada cosa) de las pantallas: landing (visor + **CTA**, botón al catálogo), catálogo en cards, ficha (galería + opción 3D), carrito, login y dashboard. Figma, Excalidraw o a mano. *Hecho cuando*: accesibles para todo el equipo.

Backlog — esperan que se libere una dependencia:

- **[setup] Configurar ESLint + Prettier compartidos** — **ESLint** (marca errores) y **Prettier** (formatea solo) con reglas compartidas para `web` y `api` desde la raíz, con scripts (`npm run lint`, `npm run format`). *Hecho cuando*: `lint` corre en los dos workspaces. (*Depende del monorepo.*)
- **[setup] Scaffold del frontend** — crear el proyecto base ("scaffold" = estructura que ya levanta, sin pantallas reales) en `apps/web`: React + Vite + TypeScript + Tailwind. Se arranca con `npm create vite@latest`. *Hecho cuando*: `npm run dev` levanta y se ven los estilos. (*Depende del monorepo.*)
- **[setup] Scaffold del backend** — crear el proyecto base en `apps/api`: NestJS + TypeScript (con `nest new`), con un **health check** (`GET /health` que devuelve 200 para chequear que el server está vivo). *Hecho cuando*: responde 200 en local. (*Depende del monorepo.*)

- **[infra] Variables de entorno + .env.example** — definir las **variables de entorno** de `web` y `api` (config que no va en el código: URLs, claves) y dejar un `.env.example` con los nombres pero sin valores reales. `.env` va en `.gitignore`. *Hecho cuando:* existe `.env.example` y `.env` está ignorado. (*Depende de los scaffolds.*)
- **[documentation] README inicial** — descripción del proyecto y pasos para levantarlo en local (requisitos como la versión de Node, instalar, correr `web` y `api`). *Hecho cuando:* alguien que clona puede levantarlo siguiendo solo el README. (*Depende del monorepo.*)
- **[visor-3d] POC del visor 3D** — una **POC** (prueba de concepto): con React Three Fiber + drei (3D en React), cargar un `.glb` del banco y rotarlo con el mouse (con **OrbitControls**, helper de drei que mueve la cámara). *Hecho cuando:* en local se ve y se rota. (*Depende del scaffold del frontend y de los modelos del banco.*)